

1. QUY TRÌNH PHÁT TRIỂN PHẦN MỀM
2. CÁC KHÁI NIỆM CƠ BẢN TRONG THIẾT KẾ HƯỚNG ĐỐI TƯỢNG
3. CÁC PHƯƠNG PHÁP PHÂN TÍCH THIẾT KẾ HƯỚNG ĐỐI TƯỢNG

1. Quy trình phát triển phần mềm

Quy trình phát triển phần mềm là một cấu trúc bao gồm tập hợp các thao tác và các kết quả tương quan sử dụng trong việc phát triển để sản xuất ra một sản phẩm phần mềm. Nhìn chung, một quy trình phát triển phần mềm bao gồm các giai đoạn như sau:

1.1. Giải pháp, yêu cầu

Nhiệm vụ: Thực hiện khảo sát chi tiết yêu cầu của khách hàng để từ đó tổng hợp vào tài liệu giải pháp. Tài liệu này phải mô tả đầy đủ các yêu cầu về chức năng, phi chức năng và giao diện.

Kết quả: Đầu ra của giai đoạn này là Tài liệu đặc tả yêu cầu

Nhiệm vụ: Thực hiện thiết kế và tổng hợp vào tài liệu thiết kế.

Kết quả: Tài liệu thiết kế tổng thể, thiết kế module, thiết kế CSDL

1.3. Lập trình

Nhiệm vụ: Lập trình viên thực hiện lập trình dựa trên tài liệu Giải pháp và Thiết kế đã được phê duyệt.

Kết quả: Source code.

1.4. Kiểm thử

Nhiệm vụ: Tester tạo kịch bản kiểm thử (test case) theo tài liệu đặc tả yêu cầu, thực hiện kiểm thử và cập nhật kết quả vào kịch bản kiểm thử, log lỗi trên các tool quản lý lỗi.

Kết quả: Test case , lỗi trên hệ thống quản lý lỗi.

1.5. Triển khai

Nhiệm vụ: Triển khai sản phẩm cho khách hàng.

Kết quả: Biên bản triển khai với khách hàng.

2. Các khái niệm cơ bản trong thiết kế hướng đối tượng

❖ Các đối tượng

Đối tượng là khái niệm cơ sở quan trọng nhất của cách tiếp cận hướng đối tượng. Đối tượng là một khái niệm, một sự trừu tượng hoá hay một sự vật có nghĩa trong bài toán đang khảo sát. Đó chính là các mục mà ta đang nghiên cứu, đang thảo luận về chúng. Đối tượng là thực thể của hệ thống, của CSDL và được xác định thông qua định danh của chúng. Thông thường các đối tượng được mô tả bởi các danh từ riêng (tên gọi) hoặc được tham chiếu tới trong các mô tả của bài toán hay trong các thảo luận với người sử dụng. Có những đối tượng là những thực thể có trong thế giới thực như người, sự vật cụ thể, hoặc là những khái niệm như một công thức, hay khái niệm trừu tượng, v.v. Có một số đối tượng được bổ sung vào hệ thống với lý do phục vụ cho việc cài đặt và có thể không có trong thực tế.

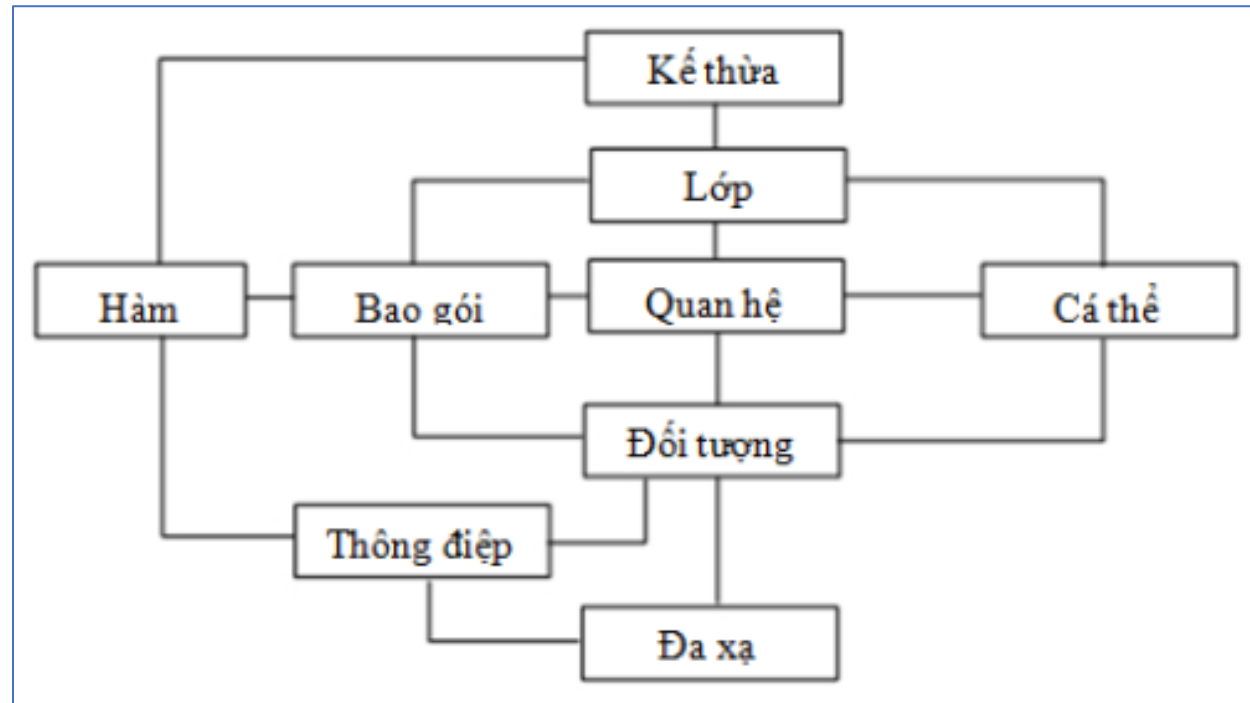
2. Các khái niệm cơ bản trong thiết kế hướng đối tượng

❖ Các đối tượng

Đối tượng là những thực thể được xác định trong thời gian hệ thống hoạt động. Trong giai đoạn phân tích, ta phải đảm bảo rằng các đối tượng đều được xác định bằng các định danh. Đến khâu thiết kế, ta phải lựa chọn cách thể hiện những định danh đó theo cách ghi địa chỉ bộ nhớ, gán các số hiệu, hay dùng tổ hợp một số giá trị của một số thuộc tính để biểu diễn. Theo quan điểm của người lập trình, đối tượng được xem như là một vùng nhớ được phân chia trong máy tính để lưu trữ dữ liệu (thuộc tính) và tập các hàm thao tác trên dữ liệu được gán với nó. Bởi vì các vùng nhớ được phân hoạch là độc lập với nhau nên các đối tượng có thể tham gia vào nhiều chương trình khác nhau mà không ảnh hưởng lẫn nhau.

2. Các khái niệm cơ bản trong thiết kế hướng đối tượng

❖ Các đối tượng



Những khái niệm cơ bản của phương pháp hướng đối tượng

2. Các khái niệm cơ bản trong thiết kế hướng đối tượng

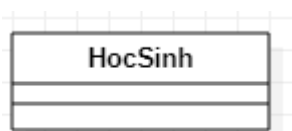
❖ Lớp đối tượng

Đối tượng là thể hiện, là một đại biểu của một lớp. *Lớp là một mô tả về một nhóm các đối tượng có những tính chất (thuộc tính) giống nhau, có chung các hành vi ứng xử (thao tác gần như nhau), có cùng mối liên quan với các đối tượng của các lớp khác và có chung ngữ nghĩa trong hệ thống.* Lớp chính là cơ chế được sử dụng để phân loại các đối tượng của một hệ thống. Lớp thường xuất hiện dưới dạng những danh từ chung trong các tài liệu mô tả bài toán hay trong các thảo luận với người sử dụng. Cũng như các đối tượng, lớp có thể là những nhóm thực thể có trong thế giới thực, cũng có những lớp là khái niệm trừu tượng và có những lớp được đưa vào trong thiết kế để phục vụ cho cài đặt hệ thống, v.v.

2. Các khái niệm cơ bản trong thiết kế hướng đối tượng

❖ Lớp đối tượng

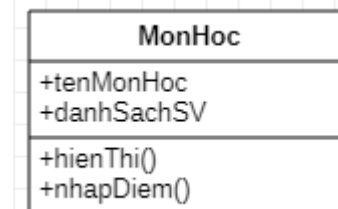
Lớp và mối quan hệ của chúng có thể mô tả trong các biểu đồ lớp biểu đồ đối tượng và một số biểu đồ khác của UML. Trong biểu đồ lớp, lớp được mô tả bằng một hình hộp chữ nhật, trong đó có tên của lớp, có thể có các thuộc tính và các hàm (phương thức)



a. Tên của lớp



b. Tên và thuộc tính



c. Tên, thuộc tính và phương thức

Các ký hiệu mô tả lớp trong UML

❖ Lớp đối tượng

Chúng ta nên đặt tên theo một quy tắc thống nhất như sau:

+ Tên của lớp thì chữ cái đầu của tất cả các từ đều viết hoa,

SinhVien, HocSinh, KhachHang, v.v.

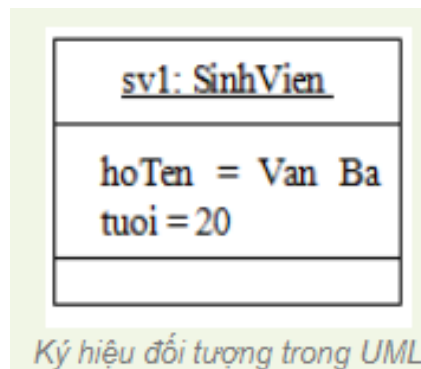
+ Tên của hàm (phương thức) viết giống như tên của đối tượng nhưng có thêm cặp ngoặc đơn '(' và ')',

hienThi(), nhapDiem(), v.v.

2. Các khái niệm cơ bản trong thiết kế hướng đối tượng

❖ Các giá trị và các thuộc tính của đối tượng

Giá trị (value) là một phần của dữ liệu. Các giá trị thường là các số hoặc là các ký tự. Thuộc tính của đối tượng là thuộc tính của lớp được mô tả bởi giá trị của mỗi đối tượng trong lớp đó.



❖ Giới thiệu

❑ SOLID - 5 nguyên lý của thiết kế hướng đối tượng

- ✓ S – Single-responsibility principle (nguyên lý đơn nhiệm)
- ✓ O – Open-closed principle (nguyên lý mở rộng - hạn chế)
- ✓ L – Liskov substitution principle (nguyên lý thay thế Liskov)
- ✓ I – Interface segregation principle (nguyên lý giao diện phân biệt - hay phân tách interface)
- ✓ D – Dependency Inversion Principle (nguyên lý nghịch đảo phụ thuộc)

S – Single-responsibility principle (nguyên lý đơn nhiệm)

- ❖ Nội dung: Một class hay method chỉ nên giữ 1 trách nhiệm duy nhất.
- ❖ Giải thích: Một class có quá nhiều chức năng cũng sẽ trở nên cồng kềnh và phức tạp. Khi yêu cầu thay đổi, một class quá cồng kềnh thì việc thay đổi code rất khó khăn và mất nhiều thời gian. Áp dụng nguyên lý đơn nhiệm chia các chức năng thành nhiều class khác nhau giúp việc quản lý, mở rộng, bảo trì code thuận tiện hơn.
- ❖ Lưu ý
 - ✓ Về bản chất nguyên lý này chỉ là hướng dẫn không phải là nguyên tắc tuyệt đối.

Có những trường hợp như các class Helper xét cho cùng toàn bộ các hàm trong class này đều thực hiện những tác vụ nhỏ nên nếu số lượng hàm ít vẫn có thể cho các hàm này vào cùng 1 class.

S – Single-responsibility principle (nguyên lý đơn nhiệm)

- ✓ Tuy nhiên khi số lượng hàm tăng lên quá nhiều thì nên cân nhắc sử dụng nguyên lý này để chia nhỏ module thuận tiện cho việc quản lý.
 - Việc hiểu và áp dụng nguyên này giúp cho việc viết code dễ đọc, dễ hiểu, dễ quản lý hơn.
- ✓ Tuy nhiên nguyên đơn nhiệm là nguyên lý đơn giản nhưng khó áp dụng đúng, việc xác định khi nào cần áp dụng khi nào không còn phụ thuộc vào việc người code xác định được đúng chức năng của module đang làm.

- Nội dung: Có thể mở rộng 1 module nhưng hạn chế sửa đổi bên trong module đó.
- Giải thích: Theo nguyên lý một module phải đáp ứng 2 điều kiện:
 - Dễ mở rộng: có thể dễ dàng nâng cấp, mở rộng, thêm tính năng mới cho module khi có yêu cầu.
 - Hạn chế sửa đổi: Hạn chế hoặc cấm việc sửa source code của module sẵn có.

Thông thường việc mở rộng thêm chức năng thì phải viết thêm code, vậy để thiết kế ra một module có thể dễ dàng mở rộng nhưng lại hạn chế sửa đổi code ta cần làm gì. Cách giải quyết là tách những phần dễ thay đổi ra khỏi phần khó thay đổi mà vẫn đảm bảo không ảnh hưởng đến phần còn lại.

Ví dụ:

- Đặt vấn đề: Ta cần 1 lớp đảm nhận việc kết nối đến CSDL. Thiết kế ban đầu chỉ có SQL Server và MySQL. Thiết kế ban đầu có dạng như sau:

```
class ConnectionManager
{
    public function doConnection(Object $connection)
    {
        if($connection instanceof SqlServer) {
            //connect with SqlServer
        } elseif($connection instanceof MySql) {
            //connect with MySql
        }
    }
}
```

Sau đó yêu cầu đặt ra phải kết nối thêm đến Oracle và một vài hệ CSDL khác. Để thêm chức năng ta phải thêm vào code những khối esleif khác, việc này làm code cồng kềnh và khó quản lý hơn.

❖ Giải pháp:

- Áp dụng Abstract thiết kế lại các lớp SqlServer, MySql, Oracle...
- Các lớp này đều có chung nhiệm vụ tạo kết nối đến csdl tương ứng có thể gọi chung là Connection.
- Cách thức kết nối đến csdl thay đổi tùy thuộc vào từng loại kết nối nhưng có thể gọi chung là doConnect.
- Vậy ta có lớp cơ sở Connection có phương thức doConnect, các lớp cụ thể là SqlServer, MySql, Oracle... kế thừa từ Connection và overwrite lại phương thức doConnect phù hợp với lớp đó.

O – Open-closed principle (OCP - nguyên lý mở rộng - hạn chế)

Thiết kế sau khi làm lại có dạng như sau:

Với thiết kế này khi cần kết nối đến 1 loại csdl mới chỉ cần thêm 1 lớp mới kế thừa Connection mà không cần sửa đổi code của lớp ConnectionManager, điều này thỏa mãn 2 điều kiện của nguyên lý OCP.

Lưu ý:

- Trên thực tế không có mô hình nào thỏa mãn hoàn toàn OCP, sẽ luôn có những thay đổi khiến mô hình không thỏa mãn OCP.

```
abstract class Connection()
{
    public abstract function doConnect();
}

class SqlServer extends Connection
{
    public function doConnect()
    {
        //connect with SqlServer
    }
}

class MySql extends Connection
{
    public function doConnect()
    {
        //connect with MySql
    }
}

class ConnectionManager
{
    public function doConnection(Connection $connection)
    {
        //something
        //.....
        //connection
        $connection->doConnect();
    }
}
```

O – Open-closed principle (OCP - nguyên lý mở rộng - hạn chế)

Ví dụ với yêu cầu show ra toàn bộ các hình được đưa vào ta có mô hình sau:

```
abstract class Shape
{
    public abstract function Draw();
}

class Square extends Shape
{
    public function Draw()
    {
        // show Square
    }
}

class Circle extends Shape
{
    public function Draw()
    {
        // show Circle
    }
}

class DrawShape
{
    public function DrawAllShape(array $shapes)
    {
        // $shapes is an array of shape object
        foreach($shapes as $shape) {
            $shape->draw();
        }
    }
}
```

Nếu các thay đổi đưa ra như thêm các hình mới thì mô hình vẫn thỏa mãn OCP. Nhưng nếu yêu cầu thay đổi là show hình tròn trước hoặc không show ra những hình đặc biệt thì chúng ta vẫn phải thay đổi code của function DrawAllShape và việc này làm thiết kế không thỏa mãn OCP.

- Khi áp dụng nguyên lý này để thiết kế cần phải xác định được những thứ dễ bị thay đổi để thiết kế phù hợp với thay đổi đó. Việc này cần kinh nghiệm và một tầm nhìn xa.

L – Liskov substitution principle (nguyên lý thay thế Liskov)

- Nội dung: Lớp B được gọi là kế thừa từ lớp A khi và chỉ khi với mọi hàm F thao tác trên các đối tượng của A, cách cư xử (behavior) của F không đổi khi thay thế các đối tượng của A bằng các đối tượng của B.
- Giải thích:
 - Nội dung nguyên lý có thể hiểu như sau trong một chương trình, các object của class con có thể thay thế class cha mà không làm thay đổi tính đúng đắn của chương trình.
 - Mỗi quan hệ IS-A (là một) thường được dùng để xác định kế thừa. Lớp B kế thừa lớp A khi B là một A, do đó B có thể thay thế hoàn toàn cho A mà không làm mất đi tính đúng đắn.

Ví dụ: Thực tế ta biết hình vuông là hình chữ nhật đặc biệt có chiều cao = chiều rộng. Vậy lớp hình vuông có phải con của lớp hình chữ nhật. Xem xét mối quan hệ kế thừa giữa hình vuông và hình chữ nhật như sau:

```
class Rectangle
{
    protected $m_width;
    protected $m_height;

    public function setWidth(int $width) {
        $this->m_width = $width;
    }

    public function setHeight(int $height) {
        $this->m_height = $height;
    }

    public function getWidth() {
        return $this->m_width;
    }

    public function getHeight() {
        return $this->m_height;
    }

    public function getArea() {
        return $this->m_width * $this->m_height;
    }
}

class Square extends Rectangle
{
    public function setWidth(int $width) {
        $this->m_width = $width;
        $this->m_height = $width;
    }

    public function setHeight(int $height) {
        $this->m_height = $height;
        $this->m_width = $height;
    }
}
```

L – Liskov substitution principle (nguyên lý thay thế Liskov)

Theo nguyên lý Liskov thì lớp Square có thể thay thế lớp Rectangle nên ta thực hiện việc test như sau:

```
class Test
{
    public function checkArea(Rectangle $r)
    {
        $r->setWidth(10);
        $r->setHeight(5);

        if($r->getArea() == 50) {
            return 'true';
        }

        return 'false';
    }
}

$test = new Test;
echo 'Test with Rectangle: ' . $test->checkArea(new Rectangle).PHP_EOL; // Test with
echo 'Test with Square: ' . $test->checkArea(new Square).PHP_EOL; // Test with Rectar
```

Phương thức test hoạt động đúng (50) khi \$r\$ là một thể hiện của Rectangle nhưng khi thay thế \$r\$ là thể hiện của Square thì kết quả là false (25). Lớp Square đã làm mất đi tính đúng đắn của chương trình.

Vậy hình vuông không phải là 1 hình chữ nhật (?). Xét về mặt hành vi thì 1 hình vuông không phải là 1 hình chữ nhật vì hành vi của hình vuông không thỏa mãn yêu cầu của hàm checkArea.

Lưu ý & kết luận:

- Trong đời sống, A là B (hình vuông là hình chữ nhật) không có nghĩa là class A nên kế thừa class B.
- Trong lập trình chỉ cho class A kế thừa class B khi class A thay thế được cho class B. Cần xem xét các đối tượng về mặt hành vi chứ không nên sử dụng những mối quan hệ trong đời thật.

I – Interface segregation principle (ISP - nguyên lý tách biệt giao tiếp hay phân tách interface)

- Nội dung: Client không nên phụ thuộc vào giao tiếp (interface) mà chúng không sử dụng.
- Giải thích:
 - Khi viết các giao tiếp (interface) chỉ nên thêm những phương thức mà nó cần phải có.
 - Nếu thêm các phương thức không cần thiết vào 1 interface các lớp thực hiện giao tiếp (class implement interface) sẽ bị bắt buộc thực thi những phương thức không cần thiết đó. Điều này dẫn đến sự dư thừa trong thực thể.

Ví dụ:

- Đặt vấn đề: Xây dựng 1 interface Phone, các lớp SmartPhone, PhoneBox implement Phone.

```
interface Phone
{
    public function call();
    public function sms();
    public function picture();
}

class SmartPhone implements Phone
{
    public function call()
    {
        // do call
    }

    public function sms()
    {
        // send sms
    }

    public function picture()
    {
        // take picture
    }
}

class PhoneBox implements Phone
{
    public function call()
    {
        // do call
    }

    public function sms()
    {
        throw new Exception("PhoneBox can't send SMS");
    }

    public function picture()
    {
        throw new Exception("PhoneBox can't take picture");
    }
}
```

Nếu lớp Phone có thêm các phương thức mới thì các lớp SmartPhone, PhoneBox sẽ phải thay đổi theo trong khi nó không cần hoặc không thể thực hiện.

■ Giải pháp:

- Chia nhỏ interface Phone thành những interface mang nhiệm vụ đặc thù
- Các lớp SmartPhone, PhoneBox implement interface đặc thù mà nó cần

```
interface Callable()
{
    public function call();
}

interface Smsable()
{
    public function sms();
}

interface Pictureable()
{
    public function sms();
}

class SmartPhone implements Callable, Smsable, Pictureable
{
    public function call()
    {
        // do call
    }

    public function sms()
    {
        // send sms
    }

    public function picture()
    {
        // take picture
    }
}

class PhoneBox implements Callable
{
    public function call()
    {
        // do call
    }
}
```

Như vậy khi mở rộng ta thêm các interface đặc thù với chức năng mở rộng thì sẽ không ảnh hưởng đến các lớp dẫn xuất.

Lưu ý & kết luận:

- Nguyên lý ISP (tách biệt giao tiếp) có mối liên hệ với nguyên lý OCP (nguyên lý mở rộng - hạn chế). Vi phạm ISP có khả năng dẫn tới sự vi phạm OCP.
- Những lớp trừu tượng chứa quá nhiều thuộc tính và chức năng gọi là những lớp bị ô nhiễm (polluted).
- Các lớp dẫn xuất phụ thuộc vào polluted interface làm tăng sự kết dính giữa các thực thể. Khi nâng cấp, sửa đổi đòi hỏi các interface này thay đổi, các lớp dẫn xuất này buộc phải thay đổi theo, điều này vi phạm nguyên lý OCP ().

•Nội dung:

- Module ở level cao hơn không nên phụ thuộc vào module ở level thấp hơn mà cả 2 nên phụ thuộc vào sự trừu tượng (abstractions).
- Sự trừu tượng (abstraction) không nên phụ thuộc vào chi tiết (details) mà ngược lại chi tiết nên phụ thuộc vào sự trừu tượng.

•Giải thích:

- Có thể hiểu nguyên lý này như sau: những thành phần trong 1 chương trình chỉ nên phụ thuộc vào những cái trừu tượng (abstraction). Những thành phần trừu tượng không nên phụ thuộc vào các thành phần mang tính cụ thể mà nên ngược lại.
- Những cái trừu tượng (abstraction) là những cái ít thay đổi và biến động, nó tập hợp những đặc tính chung nhất của những cái cụ thể. Những cái cụ thể dù khác nhau thế nào đi nữa đều tuân theo các quy tắc chung mà cái trừu tượng đã định ra. Việc phụ thuộc vào cái trừu tượng sẽ giúp chương trình linh động và thích ứng tốt với các sự thay đổi diễn ra liên tục.

Ví dụ:

- Đặt vấn đề: Ta cần 1 lớp đảm nhiệm chức năng thông báo khi nhận được 1 event từ hệ thống. Thiết kế ban đầu có dạng như sau:

```
class Notice
{
    public function pushNotification()
    {
        //push notic
    }
}

class EventListener
{
    public function listen($event)
    {
        $notice = new Notice;
        $notice->pushNotification();
    }
}
```

D – Dependency Inversion Principle (DIP - nguyên lý nghịch đảo phụ thuộc)

Như thiết kế này lớp EventListener muốn hoạt động thì phụ thuộc vào lớp Notice.

Khi yêu cầu thay đổi ta cần 1 số event thì thông báo 1 số event thì ghi lại log. Lúc này ta thêm 1 lớp **LogWriter** và sửa code của hàm **listen()**:

```
class LogWriter
{
    public function writeLog()
    {
        // write something
    }
}

class EventListener
{
    public function listen($event)
    {
        if($event == 'notice') {
            $notice = new Notice;
            $notice->pushNotification();
        } elseif($event == 'write') {
            $writer = new LogWriter;
            $writer->writeLog();
        }
    }
}
```

Như vậy với mỗi yêu cầu tăng thêm lớp EventListener sẽ càng phụ thuộc nhiều lớp, đồng thời phải sửa hàm listen() nhiều lần điều này vi phạm nguyên lý OCP (nguyên lý mở rộng - hạn chế).

❖ Giải pháp:

- Xây dựng 1 interface Handler, các lớp cụ thể Notice, LogWriter sẽ implements Handler.
- Lớp EventListener phụ thuộc vào Handler thay vì các lớp cụ thể Notice, LogWriter.

D – Dependency Inversion Principle (DIP - nguyên lý nghịch đảo phụ thuộc)

```
interface Handler
{
    public function fire();
}

class Notice implements Handler
{
    public function fire()
    {
        //push notic
    }
}

class LogWriter implements Handler
{
    public function fire()
    {
        // write something
    }
}

class EventListener
{
    public function listen(Handler $event)
    {
        $event->fire();
    }
}
```

Như vậy lớp EventListener không còn phụ thuộc vào các lớp cụ thể mà chỉ phụ thuộc vào 1 lớp trừu tượng Handler, ta có thể thêm các lớp cụ thể mà không hề ảnh hưởng đến lớp EventListener.

Lưu ý & kết luận:

- Nguyên lý DIP (nghịch đảo phụ thuộc) có mối liên hệ với nguyên lý OCP (nguyên lý mở rộng - hạn chế). Vi phạm DIP có khả năng dẫn tới sự vi phạm OCP.
- Dependency Inversion Principles != Dependency Injection. Dependency Injection chỉ là 1 trong những pattern để thực hiện Dependency Inversion Principles.

BÀI 1 — S: SINGLE RESPONSIBILITY PRINCIPLE (SRP)

Công ty EduCenter xây dựng hệ thống quản lý học viên.

Hiện tại có lớp StudentManager thực hiện các nhiệm vụ:

- Quản lý thông tin học viên
- Tính điểm trung bình
- Lưu dữ liệu xuống file
- In báo cáo

YÊU CẦU

1. Phân tích lớp StudentManager và chỉ ra:
 - Bao nhiêu trách nhiệm
 - Vi phạm SRP ở điểm nào
2. Đề xuất thiết kế lại:
 - Tách thành các lớp phù hợp
3. Vẽ Class Diagram trước và sau khi refactor

BÀI 2 –O: OPEN–CLOSED PRINCIPLE (OCP)

Công ty ShopOnline có hệ thống tính phí vận chuyển:

- Giao thường
- Giao nhanh

Hiện tại, khi thêm hình thức giao mới, lập trình viên phải sửa code cũ.

YÊU CẦU

1. Phân tích thiết kế hiện tại và chỉ ra:
 - Vì sao vi phạm OCP
2. Thiết kế lại hệ thống sao cho:
 - Có thể thêm hình thức giao mới
 - Không sửa code cũ
3. Áp dụng:
 - Interface hoặc abstract class

CÂU 1. PHÂN TÍCH LỚP StudentManager

1. Bao nhiêu trách nhiệm?

Lớp StudentManager đang có **4 trách nhiệm khác nhau**:

Trách nhiệm	Bản chất
Quản lý học viên	Nghịệp vụ (business logic)
Tính điểm trung bình	Xử lý nghiệp vụ
Lưu dữ liệu	Hạ tầng / persistence
In báo cáo	Trình bày / reporting

➔ **Mỗi trách nhiệm thuộc một mối quan tâm khác nhau (concern)**

2. Vi phạm SRP ở điểm nào?

✓ Vi phạm SRP vì:

- Lớp có nhiều hơn 1 lý do để thay đổi

Phân tích cụ thể:

Tình huống thay đổi	Có phải sửa StudentManager không?
Thay đổi cách tính điểm	Có
Thay đổi định dạng file lưu	Có
Thay đổi mẫu báo cáo	Có
Thay đổi thông tin học viên	Có

→ 4 lý do thay đổi → vi phạm SRP nghiêm trọng

CÂU 2. ĐỀ XUẤT THIẾT KẾ LẠI (REFACTOR)

✓ Nguyên tắc tách

- Mỗi lớp chỉ chịu trách nhiệm cho 1 mối quan tâm
- Mỗi lớp chỉ có 1 lý do thay đổi

✓ Thiết kế lại – Tách thành các lớp

❖ Đề xuất các lớp sau:

1. Student

- Lưu trữ thông tin học viên
- Thuộc về **domain model**

Trách nhiệm:

- id
- name
- scores

2. StudentService

- **Quản lý nghiệp vụ học viên**

Trách nhiệm:

- **Thêm / sửa / xóa học viên**
- **Không xử lý lưu file, không in báo cáo**

3. GradeCalculator

- **Tính toán điểm**

Trách nhiệm:

- **Tính điểm trung bình**
- **Có thể thay đổi thuật toán tính điểm**

4. StudentRepository

- **Lưu trữ dữ liệu**

Trách nhiệm:

- **Lưu / đọc học viên từ file (hoặc DB)**

5. StudentReportPrinter

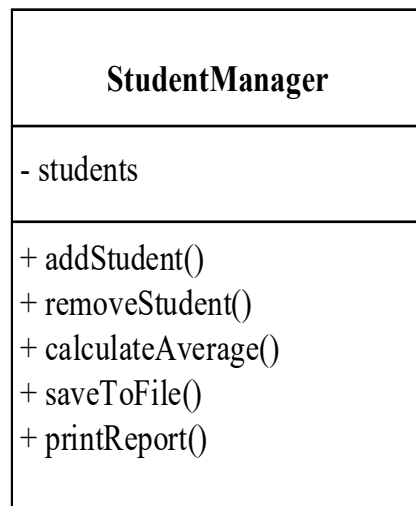
- **In báo cáo**

Trách nhiệm:

- **Xuất báo cáo (console, PDF, Excel...)**

CÂU 3. VẼ CLASS DIAGRAM (TRƯỚC & SAU)

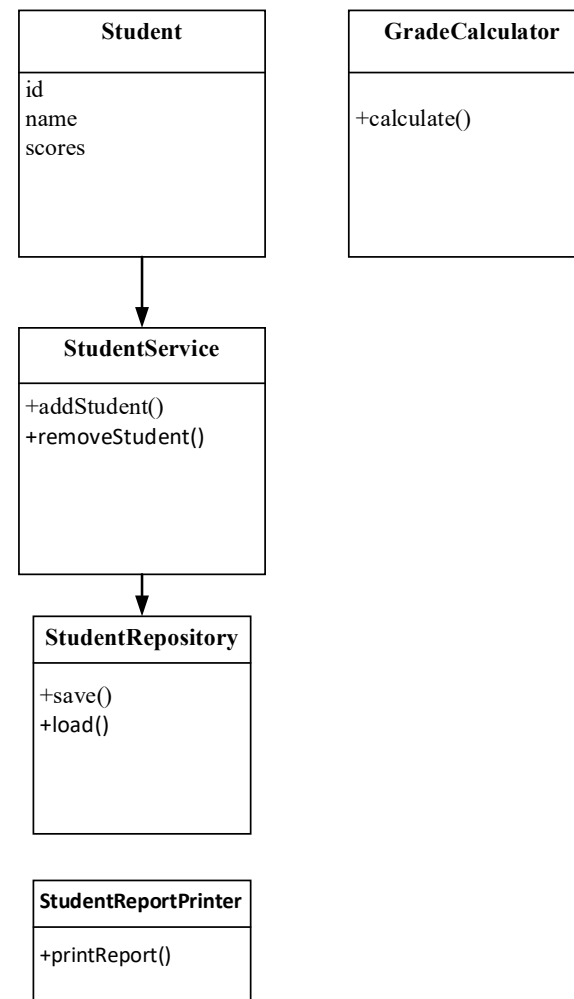
Class Diagram TRƯỚC khi refactor (vi phạm SRP)



Nhận xét:

- 1 lớp
- 4 trách nhiệm
- Rất khó mở rộng, khó bảo trì

Class Diagram SAU khi refactor (tuân thủ SRP)



CÂU 1. PHÂN TÍCH THIẾT KẾ HIỆN TẠI (VI PHẠM OCP)

✓ Thiết kế hiện tại (mô tả điển hình)

```
class ShippingFeeCalculator {  
    calculate(type, distance) {  
        if (type == "NORMAL") {  
            return distance * 5;  
        } else if (type == "FAST") {  
            return distance * 10;  
        }  
    }  
}
```

Vì sao vi phạm OCP?

✓ Phân tích:

- Khi thêm **Giao hỏa tốc**, lập trình viên phải:
 - Thêm else if
 - Sửa trực tiếp phương thức calculate()

→ **Sửa code đang chạy → vi phạm OCP**

CÂU 2. THIẾT KẾ LẠI HỆ THỐNG (TUÂN THỦ OCP)

✓ Mục tiêu:

- Thêm hình thức giao mới
- **Không sửa code cũ**
- Chỉ mở rộng bằng cách thêm class mới

✓ Ý tưởng thiết kế

- Tách **thuật toán tính phí** ra khỏi class chính
- Sử dụng **đa hình (polymorphism)**

CÂU 3. ÁP DỤNG INTERFACE / ABSTRACT CLASS

THIẾT KẾ MỚI (ĐÚNG OCP)

1. Interface ShippingStrategy

```
interface ShippingStrategy {  
    calculateFee(distance)  
}
```

Các lớp cài đặt

```
class NormalShipping implements ShippingStrategy {  
    calculateFee(distance) {  
        return distance * 5  
    }  
}
```

2. Thêm mới:

```
class ExpressShipping implements ShippingStrategy {  
    calculateFee(distance) {  
        return distance * 20  
    }  
}
```

➔ KHÔNG sửa code cũ

3. Lớp sử dụng (Client)

```
class ShippingFeeCalculator {  
    private ShippingStrategy strategy  
    setStrategy(ShippingStrategy s) {  
        strategy = s  
    }  
    calculate(distance) {  
        return strategy.calculateFee(distance)  
    }  
}
```

BÀI 3 – D: DEPENDENCY INVERSION PRINCIPLE (DIP)

Hệ thống bán hàng có lớp OrderService phụ thuộc trực tiếp vào:

- EmailService
- SMSService

Khi thay đổi cách gửi thông báo, phải sửa OrderService.

YÊU CẦU

1. Phân tích mối phụ thuộc hiện tại
2. Chỉ ra vi phạm DIP
3. Thiết kế lại hệ thống sao cho:
 - Module cấp cao không phụ thuộc module cấp thấp
 - Cả hai phụ thuộc vào abstraction
4. Áp dụng:
 - Interface
 - Dependency Injection
5. Vẽ Class Diagram

BÀI 4: TỔNG HỢP SOLID

Công ty **BookingSystem** xây dựng hệ thống:

- Đặt dịch vụ
- Thanh toán
- Gửi thông báo

YÊU CẦU

1. Thiết kế hệ thống sao cho:
 - Áp dụng ít nhất 3 nguyên lý SOLID
2. Vẽ Class Diagram
3. Giải thích rõ:
 - Mỗi nguyên lý được áp dụng ở đâu
4. Chỉ ra:
 - Nếu không dùng SOLID thì hệ thống sẽ gặp vấn đề gì

BÀI 5: Hệ thống thanh toán

Hệ thống thanh toán gồm:

- Payment
- CreditCardPayment
- CashPayment

Lớp Payment có phương thức:

`connectInternet()`

`CreditCardPayment` hoạt động bình thường.

Nhưng `CashPayment` không cần internet nên override:

`throw new`

`UnsupportedOperationException();`

YÊU CẦU

1. Phân tích quan hệ kế thừa
2. Chỉ ra phương thức override sai
3. Giải thích vì sao `CashPayment` không thể thay thế `Payment`
4. Đề xuất thiết kế mới:
 - Interface `OnlinePayment`
 - Tách chức năng kết nối internet
5. Vẽ Class Diagram mới
6. Giải thích vì sao thiết kế mới tuân thủ LSP

1. PHÂN TÍCH MỐI PHỤ THUỘC HIỆN TẠI

Thiết kế hiện tại (mô tả điển hình)

```
class OrderService {  
    EmailService emailService = new EmailService()  
    SMSService smsService = new SMSService()  
  
    placeOrder() {  
        // xử lý đơn hàng  
        emailService.send()  
        smsService.send()  
    }  
}
```

✓ Phân tích

- OrderService:
 - Tạo trực tiếp (new) EmailService
 - Tạo trực tiếp (new) SMSService
- Phụ thuộc **chặt (tight coupling)** vào:
 - Cách gửi email
 - Cách gửi SMS

✓ Kết luận yêu cầu 1

OrderService phụ thuộc trực tiếp vào các lớp cụ thể dùng để gửi thông báo.

2. CHỈ RA VI PHẠM DIP

- Vì sao vi phạm DIP?

Nguyên lý DIP	Thực tế
Module cấp cao không phụ thuộc module cấp thấp	OrderService phụ thuộc Email/SMS
Phụ thuộc abstraction	Phụ thuộc class cụ thể
Dễ thay thế	Phải sửa code cũ

Khi thêm loại thông báo mới:

- Push Notification
- Zalo / Telegram

→ Phải sửa OrderService

Kết luận yêu cầu 2

Thiết kế hiện tại vi phạm DIP vì module cấp cao **OrderService** phụ thuộc trực tiếp vào các module cấp thấp cụ thể như **EmailService** và **SMSService** thay vì phụ thuộc vào abstraction.

3. THIẾT KẾ LẠI HỆ THỐNG (TUÂN THỦ DIP)

Mục tiêu thiết kế

- OrderService **không biết** gửi bằng gì
- Chỉ biết: *có thể gửi thông báo*

4. ÁP DỤNG INTERFACE & DEPENDENCY INJECTION

THIẾT KẾ MỚI

1. Interface (Abstraction)

```
interface NotificationService {  
    send(message)  
}
```

2. Module cấp thấp (Implementation)

```
class EmailService implements NotificationService {  
    send(message) {  
        // gửi email  
    }  
}
```

```
class SMSService implements NotificationService {  
    send(message) {  
        // gửi SMS  
    }  
}
```

```
class PushNotificationService implements NotificationService {  
    send(message) {  
        // gửi push  
    }  
}
```